PLC 结构化文本代码生成与验证实验报告

摘要

本作业构建了一个面向 IEC 61131-3 Structured Text 的数据集和轻量 Agent harness，并设计了“既有 Agent × 两个基础模型”和“自研 harness × 两个基础模型”四组对照。数据集包含给定题库的 20 道公开题和 10 道原创合成题，共 30 题；每题均有接口、假设、scan-level LTL 规约和测试场景。

正式矩阵完成 30 题 × 2 个模型 × 2 个 Agent 条件的 120 格实验，四条件覆盖全部 30 题。修复 DeepSeek 的空输出后，120 格均取得非空 ST；原始 CODESYS 编译记录为 90 PASS、30 FAIL、0 TIMEOUT。后续修正编译项目适配器后，30 个原始 FAIL 源码全部复编译通过，源码未修改；因此 120 份源码均有至少一次 CODESYS 离线编译 PASS 证据，但不是使用同一版适配器统一重跑全部矩阵。首次编译为 77 PASS、39 FAIL、4 TIMEOUT；四份超时重试后，3 份通过，1 份在原始适配器下仍失败。累计项目 usage 实扣 ¥6.099742，低于 ¥18 上限。加分验证包括 STruC++ 的 P02 四份输出 20/20 自动化测试通过，以及 CBMC 对这四份翻译代码检查 918 项性质、0 项失败。CODESYS runtime 服务仍停止，112 个原设计 PLC 场景未在该 runtime 执行，PLCVerif 也未运行。结论限于本题库、模型版本和 prompt 设置。

1. 任务目标与完成范围

大作业要求完成公开题和至少 10 道自建题，为每道题编写形式规约和测试用例，设计自研 ST 代码生成 Agent，与既有 Agent 及至少两个模型开展四组实验，并报告编译、测试、验证和资源指标。

本次交付包含数据集、Agent 原型、CODESYS 离线编译适配器、完整 120 格结果、独立 ST 自动化测试和有界形式验证证据。全部 30 题均完成四条件运行；项目累计 usage 实扣 ¥6.099742，低于 ¥18 上限。Control Win x64 已安装，但 Windows 服务仍停止；因此 112 个设计场景没有在 CODESYS runtime 执行。另以独立 STruC++ 执行 P02 的 20 项测试，并用 CBMC 对 P02 的四种模型/Agent 输出检查 918 项性质；这些加分测试不替代完整 PLC runtime 或 PLCVerif 验证。

作业要求	本次产物	状态
20 道公开题与至少 10 道自建题	`dataset/tasks.json` 共 30 题	已完成
每题来源、假设、LTL 与测试场景	数据集字段及 `docs/DATASET_SPEC.md`	已完成

作业要求	本次产物	状态
自研 PLC ST Agent harness	`src/agent_harness.py`、模型适配器、CODESYS 编译器适配器	已接入 CODESYS；有限编译修复回路已实现
既有 Agent 与两种模型	Pi coding-agent 1.5.0；DMXAPI 模型 `gpt-6.1-sol`、`deepseek-v4.1-flash-1017`	四条件 × 30 题均已运行
四条件实验矩阵	`experiments/results.csv`	120/120 格完成，30 题全部四条件配对
编译、运行测试、形式验证结果	原始矩阵、后处理编译复测、STruC++ 测试、CBMC 结果	原始矩阵 90 PASS、30 FAIL；30/30 个原始 FAIL 在修正项目适配器后复编译通过；P02 独立测试 20/20、CBMC 918 项通过；112 个 CODESYS runtime 场景未执行
费用和资源记录	`experiments/*usage*.jsonl`、`results_summary.json`	234 条代理记录（233 次 HTTP 200、1 次预算拦截），项目累计实扣 ¥6.099742

2. 实验环境

项目	配置
主机环境	Windows
Python	3.12
既有 Agent	`@earendil-works/pi-coding-agent` 1.5.0；关闭工具、MCP、扩展和会话
Pi Node.js	22.19.0 side-by-side runtime；不要求改变系统默认 Node

项目	配置
API	DMXAPI OpenAI-compatible Chat Completions，经仅监听 `127.0.0.1` 的费用记录代理
模型 A	`gpt-6.1-sol`
模型 B	`deepseek-v4.1-flash-1017`
目标编译器	CODESYS V3.5 SP22 Patch 4（3.5.22.40），CODESYS Control Win V3 x64，设备 ID `0000 0004`
PLC 仿真运行时	CODESYS Control Win V3 x64 已安装（服务停止）；本报告中的 CODESYS 结果是离线编译，不是 PLC 下载或扫描仿真
形式验证	未检测到 PLCVerif；未配置 verifier adapter

费用代理只允许这两个模型 ID，并仅记录模型、token 数、供应商返回费用、耗时、HTTP 状态和时间；不记录 prompt、响应正文或 API key。密钥通过 `DMXAPI_API_KEY` 进程环境变量提供，未写入交付文件。

3. 数据集与形式规约

3.1 来源与题目覆盖

公开部分逐题取自作业提供的题库 JSON，原始 PDF 和 JSON ZIP 随包保存在 `dataset/sources/`，抽取件保存在 `dataset/question_bank_original.json`。公开题保留源题名称与接口字段；为使控制行为可重复，题目另附边界条件和 scan timing 等假设。每题的 `source` 与 `adaptation_notes` 字段指明原题来源及处理方式。

10 道自建题是原创合成，不把原始题面拼接成新题。题目覆盖泵组轮换、输送带堵塞回退、液位滞环、配方混合、风机分级、压缩机轮换、门禁停机、剔除脉冲、双罐转移和加热器诊断。设计时为每题定义 I/O、状态边界、定时条件、优先级或复位逻辑，并人工检查安全态、阈值边界和 LTL/需求的一致性。

来源	Easy	Medium	Hard	合计
给定公开题库	6	8	6	20

来源	Easy	Medium	Hard	合计
原创合成题	4	4	2	10
合计	10	12	8	30

难度按控制状态和逻辑结构定义，而不是按代码行数定义：Easy 侧重单状态、联锁、滞环或定时；Medium 涉及多步顺序、模式切换、故障恢复或数值边界；Hard 涉及队列、动态选择、阶段自动机、双设备轮换或多状态安全转移。30 题共含 279 项带方向和类型的接口字段、201 条 LTL 规约和 112 个测试场景；每题至少 3 条规约和 3 个测试场景。接口审计后，10 道自建题均补齐机器可读 I/O；自建题规范源文件为 `dataset/synthetic_tasks.json`。

3.2 Scan-level LTL 约定

规约将 PLC 每一扫描周期作为一个离散时刻：`G p` 表示每一扫描周期均满足 `p`，`X p` 表示下一扫描周期满足 `p`，`F p` 表示未来某一扫描周期满足 `p`。布尔连接符和比较式按通常逻辑解释。TON 完成、阶段状态、前一扫描值等用有名称的原子命题表示。

此表示强调周期边界上的因果和安全属性。`timerDone` 等原子需要由目标运行时模型解释；`AtMostNOf(...)`、`validIPv4` 等命名宏需在 verifier adapter 中按目标工具语法展开。由于本机没有 PLCVerif，当前 LTL 尚未交给形式化工具解析或验证；规约文本可审查不等于模型检查通过。

4. 自研 Agent 设计

Agent 使用任务 JSON 作为单一事实源，把自然语言要求、接口、假设、LTL 和测试场景拼成一个模型输入。它保存完整单题上下文，不在任务间共享状态；这样可降低跨题信息污染，但目前尚未实现长上下文压缩。模型适配器通过 OpenAI-compatible Chat Completions 调用 DMXAPI；生成结果提取为单个 ST 程序并保存。





自研 harness 的任务输入包含题面、接口和已记录假设，避免模型自行发明 I/O。CODESYS 编译器适配器以新建的 Control Win x64 项目构建生成的 PROGRAM、FUNCTION_BLOCK 或 FUNCTION，并保存完整诊断。harness 按 `repair_limit` 限制重试；成功、编译器不可用、超时或达到次数上限后停止。默认上限为 2 次修复。若目标编译器不可用，harness 只返回生成状态和轻量静态预检结果，不把静态预检失败伪装成编译结果。

静态预检只检查程序单元声明、结束标记、`VAR/END_VAR` 和少量控制结构结束标记，不进行 IEC 语法解析、类型检查或目标平台编译。真实编译由 `src/compiler.py` 中的适配器负责；真实测试还需要 PLC 仿真器或可执行的 ST runtime，本次未提供假运行结果。

Pi 基线使用相同任务 prompt 和系统规则，关闭文件、shell 等工具，并固定到相同模型接口。它与自研 harness 的资源差别是自研 harness 可以取得编译器反馈并修复，基线不修复。因此完整实验应同时比较首轮输出，并单独报告修复后结果；当前 pilot 将修复轮数置为 0。

5. 实验设计与指标

四个实验条件为 Pi + GPT、Pi + DeepSeek、自研 harness + GPT、自研 harness + DeepSeek。完整设计为每条件覆盖 30 道题，共 120 个条件-题目单元；固定温度为 0，模型 ID、题目、假设、LTL 和系统 prompt 对齐。任务运行顺序由固定随机种子打乱。正式条件允许自研 harness 最多修复两次。

主要指标定义如下：

1. 首轮编译通过率：首次 CODESYS 编译成功数除以该实验条件的任务总数；没有生成非空 ST 的任务按端到端失败计入。另报告已提交编译代码中的条件通过率，以区分生成失败和编译失败。
2. 修复后编译通过率：最终 CODESYS 编译成功数除以该实验条件的任务总数；没有生成非空 ST 的任务按端到端失败计入。另报告已提交编译代码中的条件通过率，并报告修复次数。
3. 测试通过率：通过测试场景数除以实际执行场景数；同时报告全场景通过的任务比例。
4. 形式验证：逐题记录 PASS、FAIL、TIMEOUT 和 NOT_RUN，不将未运行项算作通过。
5. 资源指标：模型 token、供应商实际费用、端到端耗时和修复次数。

6. 正式矩阵结果（累计 ¥18 费用上限）

6.1 覆盖范围

正式矩阵包含 30 题 × 2 个模型 × 2 个 Agent 条件，共 120 格。全部 120 格均已取得运行结果，30 题全部完成四条件配对；没有预算未运行格。

题目来源	完整配对	部分运行	未启动	实际运行格 / 计划格
给定公开题库	20/20	0	0	80/80
原创合成题	10/10	0	0	40/40
合计	30/30	0	0	120/120

6.2 四条件代码与编译结果

初次 DeepSeek 矩阵 60/60 格均未提取到非空 ST。诊断请求显示这些调用返回 HTTP 200，但 completion 均达到 768-token 上限。按 DeepSeek API 支持的请求参数关闭 thinking mode 后，小样本请求开始返回 `content`，完整恢复运行最终 60/60 格均提取到非空 ST。该证据支持“推理输出占满生成预算、未留下可提取正文”的原因判断；由于费用代理刻意不保存原始响应正文，无法逐格回看初次响应内容。模型接口参数和安全诊断元数据见 `experiments/config.example.json`、`src/adapters.py` 与 `experiments/deepseek_*`。

四个 DeepSeek 单元在首次 CODESYS 调用中超时。模型请求结束后使用同一 CODESYS profile 单独重试：3 份通过；P11 自研 harness 输出仍有未声明 `R_TRIG` 与不完整控制结构等编译错误。重试证据分别保存在 `experiments/codesys_validation/deepseek_timeout_retries/`，没有覆盖首次超时记录。

Agent 条件	首轮 PASS /30	最终 PASS /30	失败或超时说明
Pi + GPT-6.1 Sol	14	14	16 编译失败；基线无编译反馈修复
自研 harness + GPT-6.1 Sol	11	20	首轮 19 失败；9 格修复后通过
Pi + DeepSeek V4.1 Flash	26	28	2 编译失败；2 次超时重试均通过
自研 harness + DeepSeek V4.1 Flash	26	28	2 编译失败；超时重试一过一败；3 次修复请求

首次编译的 120 个单元共有 77 PASS、39 FAIL、4 TIMEOUT。重试结清四个超时后，原始矩阵最终记录为 90/120 PASS（75.0%）、30/120 FAIL（25.0%）、0 TIMEOUT；所有单元都有非空 ST。GPT 最终

34/60 通过，DeepSeek 最终 56/60 通过。首次结果与超时重试结果分列，避免把延时重编译误算成首次通过。

复核后发现，原 CODESYS 项目适配器把 Markdown 代码围栏留在编译输入中，未引用 Standard 3.5.22.0 库，也未把题目定义的 `ST_Pump` 和 `ST_Recipe` 添加为 DUT；此外，它把带 `VAR_IN_OUT` 的 PROGRAM 直接加入空参数任务调用。新增后处理适配器在编译视图中剥除代码围栏、引用 Standard 库、按题目定义导入 DUT，并独立编译 POU。对 30 个原始 FAIL 源文件逐一复测后，结果为 **30/30 PASS、0 FAIL、0 TIMEOUT**；每条源码 SHA-256 前后相同。故全部 120 份源码都有离线 CODESYS 编译通过证据（原始 90 PASS + 后处理 30 PASS），但后处理没有统一重跑原始 90 PASS 单元。该结果修正了失败样本的工程构建和源码抽取条件，不改变原始矩阵统计，也不验证运行时行为。逐格证据见 `experiments/posthoc_repair/recheck.csv` 和 `codesys_logs/`。

6.3 费用、资源与未执行验证

资源指标	观测值
正式矩阵 usage 费用	¥5.880938
项目全部 usage 费用（含 pilot 与诊断）	¥6.099742 / 累计上限 ¥18
GPT-6.1 Sol 项目费用	¥5.258528
DeepSeek V4.1 Flash 项目费用	¥0.841214
API usage 记录 / 状态	234 条：233 次 HTTP 200，1 次预算拦截
输入 / 输出 token	435,406 / 171,446
请求中位耗时	GPT 12,443 ms；DeepSeek 6,979 ms
原始矩阵 CODESYS 离线编译	90 PASS、30 FAIL、0 TIMEOUT
原始 FAIL 单元后处理复测	30/30 PASS；120 份源码均有至少一次编译通过证据

资源指标	观测值
CODESYS runtime 原设计场景	0/112 执行；Control Win 服务停止
STruC++ 补充功能测试	P02 四份输出共 20/20 PASS
CBMC 有界性质检查	P02 四份翻译输出共 918 项检查、0 项失败
PLCVerif / 全数据集形式验证	PLCVerif 未安装；未执行

正式矩阵占项目实际费用 ¥5.880938；含 pilot、DeepSeek 诊断与恢复等所有记录后为 ¥6.099742，低于 ¥18 上限。CODESYS runtime 测试未执行是因为 Control Win x64 服务仍停止，且项目没有 PLC runtime 测试 adapter。STruC++ 与 CBMC 的补充结果仅覆盖 P02，不能替代 112 个 runtime 场景或证明全部 30 题满足 LTL。

费用和 token 以 `experiments/*usage*.jsonl` 为准。代理日志不含 run ID，个别单元的 CSV 行级费用可能缺失或延迟，因此不据 CSV 行级费用比较单题成本。代理不保存 prompt 或响应正文，原始 DeepSeek 空响应无法逐条回看；原因诊断依据 HTTP 状态、completion token 数和受控 thinking 参数对照。

6.4 加分项：自动化测试与有界形式验证

为兑现自动化测试与形式验证加分项，选取有边界算术和状态更新的 P02，在 Pi/self × GPT/DeepSeek 四种正式输出上执行相同扫描周期断言。每份代码运行 5 项测试（数据集 3 个场景，加 32 位全升沿和全降沿边界），由 STruC++ 0.7.0 编译并运行，结果为 20/20 PASS。DeepSeek Pi 输出采用等价表达式归一化以适配 STruC++ 0.7.0；原始 ST 未改，转换和源码哈希均记录在结果 JSON。

随后 CBMC 5.95.1 对这四份 P02 的 STruC++ 生成代码检查两个任意连续 32 位输入变化，并检查边沿掩码、changed/edge 标志、计数、前值更新、循环展开、边界、指针及未定义移位等性质，合计 918 项通过、0 项失败。该证明范围限于 P02 的 STruC++ 翻译代码及所列有界状态，不是 CODESYS runtime 或所有 PLC 环境的等价证明。复现脚本、测试模板、CBMC harness、运行日志、哈希和逐输出结果位于 `tests/structured_text/`、`tools/run_st_tests.py`、`tools/run_p02_cbmc.py` 和 `experiments/functional_tests/`、`experiments/formal_verification/`。

7. 有效性边界与后续工作

本次四条件矩阵覆盖全部 30 道题，DeepSeek 的 60 格经请求参数修正后均取得非空 ST。原始矩阵下，自研 harness 的 GPT 条件 20/30 通过、Pi GPT 基线 14/30；两种 DeepSeek 条件各 28/30。随后，修正项目适配器复测全部 30 个原始失败样本并通过；与原始通过日志合并后，120 份源文件均有至少一次离线 CODESYS 编译 PASS 记录，但原始矩阵的 90/120 首次/重试口径不变。由于模型输出、Agent 能力、有限修复和编译结果共同影响观测值，且题量有限，这些结果不能支持模型或框架的一般优劣结论。

CODESYS 离线构建不等于 PLC 下载、扫描执行或现场功能测试；I/O 映射与 timer 语义尚未由 runtime 校准。STruC++ 和 CBMC 加分检查只覆盖 P02，PLC runtime 场景执行数为 0/112，PLCVerif 执行数为 0，因此不能声称全数据集功能正确或满足全部 LTL。

后续工作可以按以下顺序推进：

1. 以管理员权限启动 CODESYS Control Win x64 服务，为 PLC runtime 实现测试 adapter，逐题执行 test_cases 并保存输入、输出和原始日志。
2. 安装 PLCVerif 并配置 adapter；把 LTL 命名宏展开为目标工具可接受的语法，检查状态和 timer 语义后逐题验证。
3. 将 STruC++ 的测试扩展到更多任务，并实现能把数据集扫描周期场景输入 CODESYS Control Win、记录 I/O 与周期状态的 runtime test adapter。
4. 安装 PLCVerif，适配 scan-level LTL 和 timer 语义，验证全部 30 题；扩大题型和模型版本后再评估一般化结论。

正式矩阵每次输出限制为 768 token，自研 harness 最多修复两次。DeepSeek 将 thinking mode 关闭后恢复了 60 格非空输出；CODESYS 最终编译超时已通过独立重试结清。项目累计 usage 实扣 ¥6.099742 / ¥18，结果表 120/120 完成，没有空响应或未结清编译超时。

附录 A 交付结构

数据集的全部 30 道题、279 项接口字段、201 条形式规约和 112 个测试场景见 `dataset/tasks.json`。实验协议见 `experiments/protocol.md`；四条件 120 格原始结果及汇总见 `experiments/results.csv` 和 `experiments/results_summary.json`；失败单元的后处理复测见 `experiments/posthoc_repair/`，可复用 DUT 定义见 `st/support_types/`；CODESYS 超时重试证据见 `experiments/codesys_validation/deepseek_timeout_retries/`。STruC++ 与 CBMC 的逐输出结果、源码哈希、测试模板、转换证据和日志分别见 `experiments/functional_tests/`、`experiments/formal_verification/`、`tests/structured_text/`。正式代码见 `st/runs/`。usage 记录仅含模型、token、费用、耗时、HTTP 状态和时间，不含 prompt、响应正文或 API key。

参考资料

1. 课程大作业说明及随附公开题库：`dataset/sources/作业说明中文原件.docx`、`dataset/sources/题库原始.pdf` 和 `dataset/sources/题库原始JSON.zip`。
2. DMXAPI OpenAI-compatible Chat Completions 与费用字段说明：https://doc.dmxapi.cn/openai-chat.html、https://doc.dmxapi.cn/dmxapi-cost.html。
3. Pi coding-agent 自定义 OpenAI-compatible provider 与 CLI 文档：https://pi.dev/docs/latest/custom-provider、https://pi.dev/docs/latest/cli。
4. CODESYS 命令行和 ScriptEngine 文档：https://content.helpme-codesys.com/en/CODESYS%20Development%20System/_cds_commandline.html、https://content.helpme-codesys.com/en/ScriptingEngine/ScriptIecLanguageObjectContainer.html、https://content.helpme-codesys.com/en/ScriptingEngine/ScriptApplication.html。
5. DeepSeek Thinking Mode 参数及 reasoning/content 返回字段：https://api-docs.deepseek.com/guides/thinking_mode/。
6. STruC++ 自动化测试说明：https://github.com/Autonomy-Logic/STruCpp/blob/development/docs/TESTING.md；CBMC 有界模型检查器：https://github.com/diffblue/cbmc。